

Universidad Politécnica de Victoria

Ingeniería en Tecnologías de la Información

Análisis Técnico de la Máquina de Estados Finitos (FSM) de la Máquina Expendedora

Materia: Fundamentos de Inteligencia Artificial

Plataforma: Arduino Mega 2560 (firmware C++, PlatformIO)

Arturo Zapata · Fabián Ramírez · Equipo

2 de octubre de 2026

Índice

1. Introducción y Contexto del Proyecto	2
2. Punto de Entrada: main.cpp	2
2.1. Arquitectura de Super-Loop	2
2.2. Inyección de Dependencias	3
3. La Máquina de Estados: VmFsm	3
3.1. Concepto y Rol	3
3.2. Mecanismo de Transición	3
3.3. Temporización No Bloqueante	3
3.4. Interrelación con las Demás Clases	4
4. Cálculo de Cambio: VmChangeCalculator	4
4.1. Denominaciones Aceptadas	4
4.2. Algoritmo Voraz con Retroceso	5
4.3. Visualización del Cambio en Carrusel	5
4.4. Decisiones de Diseño	6
5. Los 17 Estados de la FSM	8
5.1. Tabla Resumen	8
5.2. Descripción Detallada de Cada Estado	9
6. Diagrama de Maquina de Estados	10
6.1. Tabla de transiciones	12
7. Conclusiones	13

1. Introducción y Contexto del Proyecto

La máquina expendedora es un sistema embebido autónomo implementado en un **Arduino Mega 2560**. Todo el firmware corre en un único microcontrolador sobre un *super-loop* no bloqueante.

Los periféricos que gestiona el sistema son:

- **Teclado matricial 4×4** (filas D22–D25, columnas D30, D31, D33 y D29) — entrada del usuario.
- **LCD 20×4 I²C** (0x27) — retroalimentación visual.
- **Motores DC** vía PCA9685 (I²C) — despacho de producto.
- **Lector RFID MFRC522** (SPI: D53, D8) — pago con tarjeta.
- **Barrera óptica de caída** (D2, pull-up) — confirmación de entrega del motor.
- **EEPROM interna** — persistencia de inventario y caja.

El núcleo del firmware es el módulo `VmFsm`, una Máquina de Estados Finitos (FSM) con 17 estados que encapsula toda la lógica de negocio: selección de producto, cobro, despacho y cálculo de cambio.

2. Punto de Entrada: `main.cpp`

2.1. Arquitectura de Super-Loop

El firmware sigue el patrón *bare-metal super-loop*: `setup()` inicializa los periféricos una sola vez y `loop()` se ejecuta continuamente sin usar interrupciones ni RTOS.

```
1 void setup() {
2     Wire.begin(); // I2C antes del LCD y PCA9685
3     pinMode(VM_PIN_BARRIER, INPUT_PULLUP);
4     display.begin();
5     motor.begin();
6     rfid.begin();
7     fsm.begin(); // carga EEPROM y entra al estado
8     S0
9 }
10 void loop() {
11     char key = keypad.getKey(); // sondeo no bloqueante del
12     teclado
13     if (key != NO_KEY)
14         fsm.handleKey(key); // despachar tecla a la FSM
15     fsm.update(); // avanzar la maquina de estados
16 }
```

Listing 1: Núcleo del super-loop en `main.cpp`

2.2. Inyección de Dependencias

Todos los subsistemas se instancian como objetos estáticos globales y se *inyectan* al constructor de VmFsm:

```
1 static VmDisplay      display;
2 static VmEepromData   storage;
3 static VmMotorController motor;
4 static VmRfid          rfid;
5
6 static VmFsm fsm(storage, motor, rfid, onDisplay);
```

Listing 2: Instanciación e inyección en main.cpp

La función `onDisplay()` actúa como *callback* de presentación: la FSM nunca accede directamente al LCD; simplemente llama al puntero de función `DisplayFn` que le fue entregado en construcción.

3. La Máquina de Estados: VmFsm

3.1. Concepto y Rol

VmFsm implementa el patrón **Mediador/Controlador**: centraliza la lógica de negocio y coordina a los demás módulos sin que éstos se conozcan entre sí. Es una **Máquina de Moore**: las salidas (pantallas, motor, cambio) dependen únicamente del estado activo, y las transiciones dependen de eventos (teclas, RFID, temporizadores, barrera óptica).

3.2. Mecanismo de Transición

1. `enterState(FsmState next)` — cambia `_state` y llama al gancho `onEnter*()` correspondiente.
2. `update()` — llamado cada ciclo desde `loop()`. Sondea temporizadores, RFID y la barrera óptica para generar transiciones autónomas.
3. `handleKey()` — traduce el carácter físico a una acción semántica y la delega al procesador del estado activo.

3.3. Temporización No Bloqueante

Todos los temporizadores se implementan comparando `millis()` contra una referencia guardada, sin usar `delay()`:

```
1 _motorTimer = millis();                                // arrancar
2
3 // En update():
4 if ((uint32_t)(millis() - _motorTimer) >= VM_TIMEOUT_MOTOR_MS)
5     // ... accion de timeout
```

Listing 3: Patrón de temporizador no bloqueante

Constante	Valor	Uso
VM_TIMEOUT_INACTIVITY_MS	180 000 ms	Regreso a reposo en S3–S6
VM_TIMEOUT_MOTOR_MS	10 000 ms	Tiempo máximo de giro del motor en S8
VM_CAROUSEL_INTERVAL_MS	2 000 ms	Rotación de subpantalla del carrusel

Cuadro 1: Temporizadores de la FSM

3.4. Interrelación con las Demás Clases

Clase	Responsabilidad
VmEepromData	Persistencia de slots (nombre, precio, stock) y caja de efectivo; valida los datos al arrancar y los reinicializa si no son válidos.
VmMotorController	Controla motores DC vía PCA9685; detecta entrega mediante barrera óptica (D2).
VmRfid	Lectura de tarjetas MIFARE, cobro del saldo y devolución del saldo (reselecciona la tarjeta y verifica su UID).
VmKeypad	Interpreta caracteres del teclado 4×4 en acciones semánticas según el contexto.
VmDisplay	Abstracción del LCD 20×4 I ² C; recibe 4 cadenas de texto.
VmCarousel	Rota subpantallas cada 2 s llamando al callback de display.
VmChangeCalculator	Cálculo de cambio y reembolso sobre las monedas de la caja (voraz con retroceso).

Cuadro 2: Módulos del firmware y sus responsabilidades

4. Cálculo de Cambio: VmChangeCalculator

4.1. Denominaciones Aceptadas

El sistema maneja cuatro denominaciones de moneda mexicana (máximo \$10):

\$1.00 (100c) \$2.00 (200c) \$5.00 (500c) \$10.00 (1000c)

En el teclado, la correspondencia es: 1=\$1, 2=\$2, 3=\$5, 4=\$10. Las teclas 5–9 se ignoran.

4.2. Algoritmo Voraz con Retroceso

`calculate(pagado, precio, caja, resultado)` obtiene el monto pagado - precio. Con `precio = 0` calcula el reembolso de lo pagado (\$15). El algoritmo prueba primero la mayor cantidad posible de la moneda más grande y, si el resto no se puede completar con las monedas restantes, retrocede y usa una pieza menos. Así se encuentra una solución cada vez que existe (por ejemplo, \$6 con una moneda de \$5, sin monedas de \$1 y tres de \$2 se devuelve como $3 \times \$2$).

```
1 static bool findCoins(uint32_t total, uint8_t idx,
2                     const uint32_t avail[4], uint32_t use[4]) {
3     uint32_t denom = DENOMS[idx];
4     if (idx == 3) { // ultima: resto exacto
5         if (total % denom != 0 || total / denom > avail[idx]) return
6             false;
7         use[idx] = total / denom;
8         return true;
9     }
10    uint32_t qty = min(total / denom, avail[idx]);
11    for (;;) {
12        use[idx] = qty;
13        if (findCoins(total - qty * denom, idx + 1, avail, use))
14            return true;
15        if (qty == 0) return false;
16        qty--;
```

Listing 4: Búsqueda de monedas en `VmChangeCalculator`

La caja **solo se modifica si el monto se puede formar**: primero se verifica con `findCoins()` y después se llama a `deductCoins()` por cada denominación usada. Si no es posible, devuelve `false` y la caja queda intacta.

4.3. Visualización del Cambio en Carrusel

Tras un pago exitoso en efectivo, la FSM construye un **carrusel de slides**: un slide inicial de éxito seguido de uno por cada denominación no nula (de mayor a menor). Las denominaciones con cero monedas se omiten:

```
1 _carousel.addScreen(finSuccessTrampoline); // slide 0: Compra
2     Exitosa
3 const uint32_t denoms[4] = {1000u, 500u, 200u, 100u};
4 for (uint8_t i = 0; i < 4; i++) {
5     if (qtys[i] > 0u) {
6         _changeDenoms[_changeSlides] = denoms[i];
7         _changeQtys[_changeSlides] = qtys[i];
8         _changeSlides++;
9         _carousel.addScreen(changeScreenTrampoline);
10    }
11 }
```

Listing 5: Construcción dinámica del carrusel de cambio en `onEnterPantallaFin()`

Ejemplo: si el cambio es \$7.00, el carrusel muestra tres slides: *Compra Exitosa* → *\$5.00*

$\times 1 \text{ pieza} \rightarrow \$2.00 \times 1 \text{ pieza}.$

4.4. Decisiones de Diseño

- **Aritmética entera en centavos:** sin errores de coma flotante en AVR de 8 bits.
- **Descuento inmediato en EEPROM:** cada `deductCoins()` persiste al instante.
- **Retorno false si no hay monedas:** la FSM registra el monto como adeudo y muestra un mensaje (S13) hasta que el usuario pulsa [A].
- **Monedas insertadas:** cada moneda de S5 se suma a la caja al momento de insertarse; por eso el reembolso se puede formar con ellas.

5. Los 17 Estados de la FSM

5.1. Tabla Resumen

ID	Nombre	Propósito	Transiciones de salida
S_0	S0_START	Inicialización; muestra mensaje de bienvenida.	$\rightarrow S_2$ inmediatamente.
S_1	S1_INTERNAL_ERROR	Error crítico de EEPROM; sistema detenido.	Estado terminal.
S_2	S2_STANDBY	Carrusel del catálogo (una pantalla por producto).	$\rightarrow S_3$ (tecla 1–4).
S_3	S3_SELECT_CHANNEL	Confirmar producto elegido.	$\rightarrow S_4$ [A] / S_2 [*] / timeout.
S_4	S4_SELECT_PAYMENT	Elegir método: efectivo o RFID.	$\rightarrow S_5$ [A] / S_6 [B] / S_2 [*].
S_5	S5_WAIT_CASH	Acumular monedas (teclas 1–4: \$1/\$2/\$5/\$10).	$\rightarrow S_7$ (monto $\geq$ precio) / S_{15} ([*]/timeout, monto > 0) / S_2 ([*]/timeout, monto = 0).
S_6	S6_WAIT_RFID	Sondear RFID y cobrar saldo.	$\rightarrow S_7$ (OK) / S_{13} (error de lectura o saldo) / S_2 ([B], [*] o timeout).
S_7	S7_PREPARING_VEND	Reservar stock y arrancar motor.	$\rightarrow S_8$ (OK) / S_{10} (sin stock o motor rechazó el arranque).
S_8	S8_DISPENSING	Motor activo; esperar barrera óptica.	$\rightarrow S_9$ (barrera OK) / S_{10} (timeout 10 s).
S_9	S9_CONFIRMED	Entrega confirmada; esperar 1 s.	$\rightarrow S_{11}$ (efectivo) / S_{12} (RFID).
S_{10}	S10_VEND_ERROR	Rollback de stock; mensaje de error 2.5 s.	$\rightarrow S_{14}$ (RFID) / S_{15} (efectivo).
S_{11}	S11_CALC_CHANGE	Algoritmo greedy; preparar slides de cambio.	$\rightarrow S_{12}$ (éxito) / S_{13} (M4).
S_{12}	S12_FINISH_SCREEN	Carrusel: éxito + una diapositiva por denominación no nula.	$\rightarrow S_2$.
S_{13}	S13_MESSAGE_PROMPT	Estado transitorio no bloqueante para mensajes (espera A).	Depende del conector M1–M5.
S_{14}	S14_REFUND_RFID	Devolver monto cobrado a la tarjeta RFID.	$\rightarrow S_{16}$ (éxito) / S_{15} (falla).
S_{15}	S15_REFUND_CALC	Calcular devolución de monto usando monedas.	$\rightarrow S_{16}$ (éxito) / S_{13} (M5).
S_{16}	S16_REFUND_FINISH	Carrusel: operación cancelada + detalle de reembolso.	$\rightarrow S_2$.

Cuadro 3: Los 17 estados de la FSM de la máquina expendedora

5.2. Descripción Detallada de Cada Estado

S_0 — **Arranque.** Muestra mensaje de bienvenida; en el primer ciclo de `update()` transita a S_2 . Antes, `VmEepromData::begin()` carga la EEPROM y la valida (denominaciones de la caja, precios, stock y capacidad); si no hay datos válidos o cambió la versión, se reinicializa con los datos semilla.

S_1 — **Falla Interna.** Estado terminal por error de EEPROM. Requiere reinicio físico del sistema.

S_2 — **Reposo.** Carrusel con una pantalla por producto (canales 1 a 4, cada 2 s) y una quinta pantalla de adeudo si hay un adeudo pendiente.

S_3 — **Selección de Canal.** Consulta `getSlot()` para verificar que el slot está habilitado. Muestra nombre y precio del producto.

S_4 — **Selección de Pago.** El usuario elige la pasarela de cobro. Si RFID no está disponible, vuelve a esta pantalla con un aviso.

S_5 — **Espera de Efectivo.** Teclas 1–4 representan monedas de \$1, \$2, \$5 y \$10. Cada pulsación llama a `addCoins()`; si la caja la registra, se actualiza el acumulado (si la caja está llena, la moneda se rechaza). Al cubrir el precio transita automáticamente a S_7 . La tecla * (o la inactividad) cancela: con monto mayor a cero pasa a S_{15} para devolver las monedas. La pantalla muestra lo insertado, lo que falta y las teclas válidas.

S_6 — **Espera de RFID.** `VmRfid::poll()` se llama en cada ciclo sin bloquear. Saldo suficiente $\rightarrow$ `deductBalance()` $\rightarrow$ S_7 . Saldo insuficiente o error de lectura $\rightarrow$ mensaje y S_2 .

S_7 — **Reservada.** Punto de no retorno transaccional: (1) `reserveStock()` decrementa el inventario en EEPROM; (2) `motor.start()` verifica que la barrera óptica esté libre y enciende el motor. Si `start()` falla, transita a S_{10} .

S_8 — **Dispensando.** Supervisa `motor.isBusy()` en cada ciclo. La barrera óptica al detectar la caída del producto detiene el motor y reporta `VM_RESULT_DELIVERED`. Un timeout de 10 s protege contra atascos.

S_9 — **Confirmada.** Muestra pantalla de éxito 1 s. Luego, si el pago fue en efectivo, avanza a S_{11} ; si fue RFID (sin cambio), a S_{12} .

S_{10} — **Falla de Despacho.** Si `_stockReserved` es `true`, llama a `releaseStock()` para devolver la unidad al inventario. Muestra el aviso de falla 2.5 s y pasa al reembolso: S_{14} si se pagó con RFID o S_{15} si se pagó en efectivo.

S_{11} — **Cálculo de Cambio.** Invoca `VmChangeCalculator::calculate()` con el monto insertado y el precio. En caso de éxito, S_{12} construye los arrays `_changeDenoms[]` y `_changeQtys[]` para el carrusel. Si no hay monedas suficientes, registra el adeudo y muestra un aviso en S_{13} que espera la tecla [A].

S_{12} — **Pantalla Final.** El carrusel muestra: slide de éxito, seguido de un slide por cada denominación no nula (de \$10 a \$1) o el resumen de cobro RFID. Regresa a S_2 al completar todos los slides + 1 s.

S_{13} — **Prompt de Mensaje.** Estado transitorio no bloqueante para mostrar mensajes al usuario y esperar la confirmación. Transita al estado destino configurado tras pulsar la tecla A.

S_{14} — **Reembolso a Tarjeta RFID.** Intenta devolver el monto cobrado a la tarjeta RFID tras un fallo de despacho. Como el cobro dejó la tarjeta en estado `HALT`, `creditBalance()` la vuelve a seleccionar (`PICC_WakeupA`) y verifica que su UID sea el mismo antes de escribir; por eso la tarjeta debe seguir sobre el lector. Si la escritura es exitosa, transita a S_{16} ; si falla, pasa a S_{15} como contingencia.

S_{15} — **Cálculo de Reembolso en Efectivo.** Llama a `calculate()` con precio 0 para devolver la cantidad insertada con monedas de la caja. Si hay monedas suficientes, las descuenta y pasa a S_{16} ; de lo contrario, registra un adeudo interno y usa S_{13} para mostrar un mensaje al usuario.

S_{16} — **Pantalla Final de Reembolso.** Construye un carrusel que notifica la operación cancelada y desglosa el dinero devuelto (saldo en tarjeta o monedas). Al finalizar, retorna a S_2 .

6. Diagrama de Maquina de Estados

La máquina de estados tiene 17 estados (S_0 – S_{16}). Cada estado indica su condición de entrada (*Entra*), la acción que ejecuta al entrar (*Acción*) y todas sus salidas (*Sal*). La Figura 1 presenta el diagrama completo en la siguiente página y la Tabla 4 lista cada transición con su condición de guarda y su acción asociada.

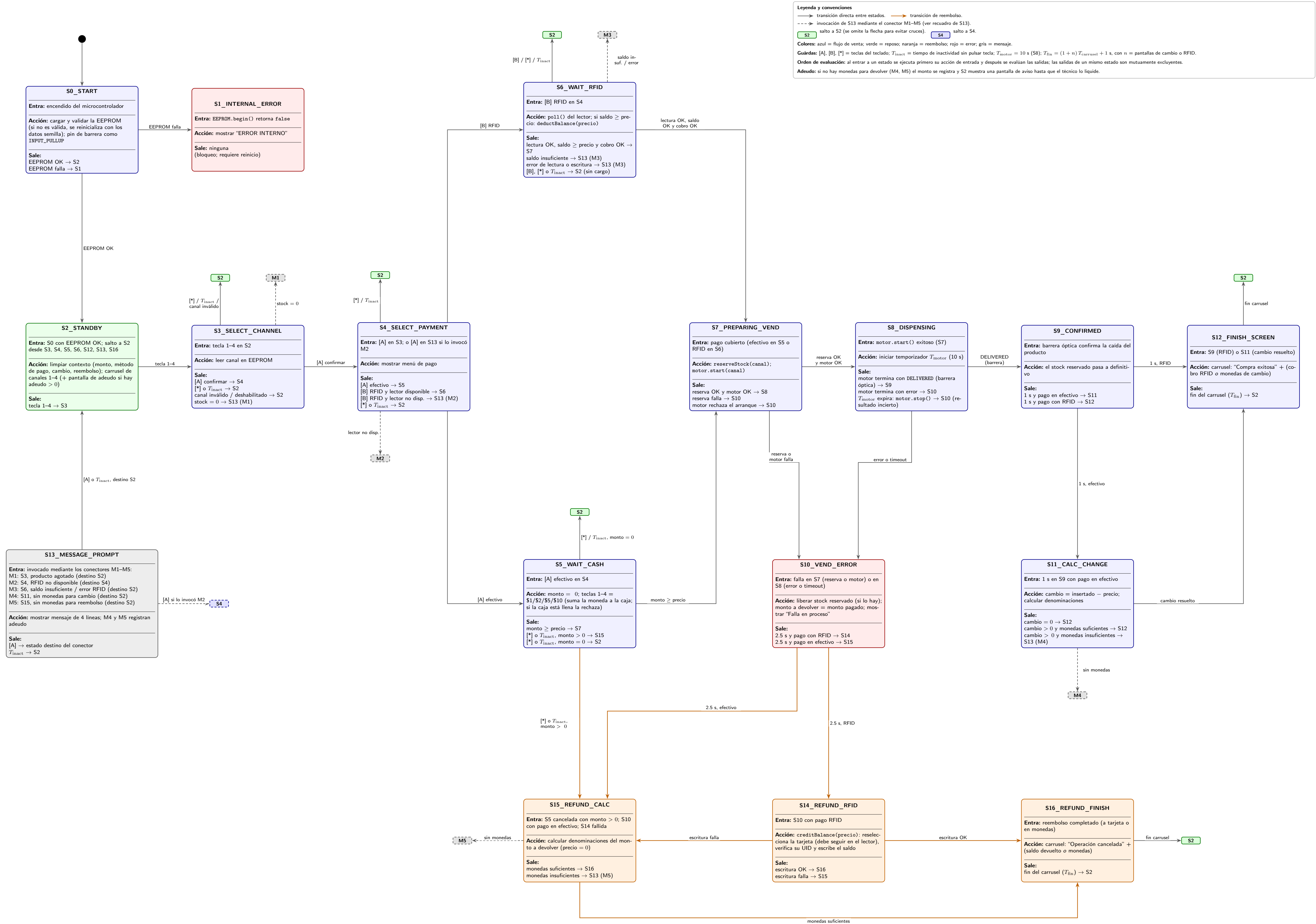


Figura 1: Diagrama de estados UML de la FSM de la máquina expendedora (17 estados). Se distinguen el flujo de venta (azul) y el flujo de reembolso (naranja); los saltos a S2 y las invocaciones de S13 se muestran con conectores para evitar cruces de flechas.

6.1. Tabla de transiciones

La Tabla 4 detalla cada transición del diagrama. Una misma fila describe un único evento; las condiciones de un mismo estado de origen son mutuamente excluyentes.

Cuadro 4: Transiciones de la FSM.

Origen	Evento / condición de guarda	Destino	Acción asociada
S0	EEPROM.begin() = true	S2	configurar pin de barrera
S0	EEPROM.begin() = false	S1	—
S1	(sin salidas)	—	bloqueo del sistema
S2	tecla 1–4	S3	seleccionar canal
S3	canal inválido o deshabilitado	S2	—
S3	stock = 0	S13 (M1)	mensaje “Producto agotado”; destino S2
S3	[A] confirmar	S4	—
S3	[*] o T_{inact}	S2	—
S4	[A] efectivo	S5	—
S4	[B] RFID y lector disponible	S6	—
S4	[B] RFID y lector no disponible	S13 (M2)	mensaje “RFID no disponible”; destino S4
S4	[*] o T_{inact}	S2	—
S5	monto $\geq$ precio	S7	—
S5	[*] o T_{inact} con monto > 0	S15	monto a devolver = monto insertado
S5	[*] o T_{inact} con monto = 0	S2	—
S6	lectura OK, saldo $\geq$ precio y deductBalance OK	S7	método = RFID; monto = precio; saldo restante = saldo – precio
S6	lectura OK y saldo $<$ precio	S13 (M3)	mensaje “Saldo insuficiente”; destino S2
S6	deductBalance falla	S13 (M3)	mensaje “Error de escritura”; destino S2
S6	error de autenticación o lectura	S13 (M3)	mensaje “Error al leer”; destino S2
S6	[B], [*] o T_{inact}	S2	sin cargo a la tarjeta
S7	reserva OK y motor.start OK	S8	—
S7	reserveStock falla	S10	resultado = rechazado antes de mover
S7	motor.start falla	S10	resultado = rechazado antes de mover
S8	motor termina con DELIVERED	S9	—
S8	motor termina con error	S10	resultado = código del motor
S8	motor ocupado y T_{motor} expiró	S10	motor.stop(); resultado = incierto
S9	1 s y pago en efectivo	S11	—
S9	1 s y pago con RFID	S12	cambio = 0
S10	2.5 s y pago con RFID	S14	monto a devolver = precio

Origen	Evento / condición de guarda	Destino	Acción asociada
S10	2.5 s y pago en efectivo	S15	monto a devolver = monto insertado
S11	cambio = 0	S12	—
S11	cambio > 0 y monedas suficientes	S12	descontar monedas del inventario
S11	cambio > 0 y monedas insuficientes	S13 (M4)	registrar adeudo; destino S2
S12	fin del carrusel (T_{fin})	S2	—
S13	[A]	destino	destino = el del conector M1–M5
S13	T_{inact}	S2	—
S14	creditBalance OK	S16	saldo restante = saldo + monto
S14	creditBalance falla	S15	monto a devolver = precio (en monedas)
S15	monedas suficientes	S16	descontar monedas del inventario
S15	monedas insuficientes	S13 (M5)	registrar adeudo; destino S2
S16	fin del carrusel (T_{fin})	S2	—

7. Conclusiones

El diseño de la FSM cumple los objetivos de un sistema embebido robusto y legible:

1. **Legibilidad lineal:** los 17 estados están numerados S_0 – S_{16} y el código fuente sigue el mismo orden de arriba a abajo, facilitando la lectura y el mantenimiento.
2. **Sin sobreingeniería:** cada clase tiene una responsabilidad única. Se eliminaron el registro transaccional (EEPROM) y el sensor de puerta, que no corresponden al hardware real actual.
3. **Visualización correcta del cambio:** el carrusel de S_{12} muestra dinámicamente solo las denominaciones no nulas, de mayor a menor, una por slide, facilitando la lectura al usuario.
4. **Denominaciones reales:** el sistema acepta exclusivamente monedas de \$1, \$2, \$5 y \$10 (teclas 1–4), coherente con el hardware de la caja real.
5. **Temporización no bloqueante:** todos los delays se implementan con `millis()`, permitiendo que el super-loop permanezca siempre receptivo a eventos externos.
6. **Carrusel corregido:** el módulo `VmCarousel` incrementa correctamente el índice en cada ciclo y muestra la primera slide de forma inmediata al entrar a un nuevo estado.